

Elisa Beihofer

Dialogue Systems 2

Lab 1 / VG-RAG

Does RAG work well in case the dialogue unfolds, and not just for one-shot queries?

I have implemented the regular part of Lab 1 in such a way that Qdrant only ever sees the latest utterance for information retrieval:

```
input: ({ context }) => ({  
    query: context.lastResult![0].utterance,  
}),
```

The latest utterance is embedded and Qdrant is searched using that embedding. It will then return the top 3 most relevant chunks and add them to the system prompt. Finally, the LLM generates the final response.

This means that when I ask a follow-up question to the system, Qdrant will once again only be queried using the current latest utterance. The LLM, however, receives the full dialogue history. So if I ask: “What is the phone number for Feelgood?”, the system follows it up with giving me the phone number. If I then continue by asking: “What are their opening hours?” and: “What is their e-mail address?”, my conversation history will know that “their” refers to Feelgood, but Qdrant will not.

I tested this case. While the LLM gave coherent responses, the retrieved data from Qdrant did not match my context-dependent questions. For the opening hours, it returned e.g. the opening hours of different service centers, but not Feelgood’s. For the e-mail question it was similar. Therefore, the LLM ended up saying that it did not have enough information / did not find the information to answer the question. Since the answers could have actually been found in the database if Qdrant had received proper context, the final answers in my test do not satisfy in real-life conditions.

My baseline system therefore works well for one-shot queries, but it fails for ambiguous follow up questions that reference earlier context. That is why I tested another condition that gives Qdrant not only the current, but also the previous user utterance.

In order to preserve the two most recent user turns, I changed my code from above to filter the context for only messages from the user. I then sliced the user messages to only take the two most recent ones and combined them into one query to send to the embedding model / Qdrant.

With the slightly extended context, the system was able to answer my first two test questions. As earlier, Qdrant received all the necessary context from the question itself. For the second previous-context-dependent opening hour question, the extended

context for the query helped preserve the necessary information (“Feelgood” instead of just “their”) and the LLM was able to answer the question. However, Qdrant also retrieved information about disability support that was unrelated and the LLM mixed it into its generated text. On the third question, the system failed again. With the context only being extended by one user turn, “Feelgood” was not included in the query anymore. This led to Qdrant not returning any information related to Feelgood and the LLM answering with the wrong e-mail address, namely the one for the disability support.

So while the extra context helped initially, it already distorted the information that was given on the second turn by providing additional unrelated information. It then failed on the third turn, when the two most recent turns did not provide sufficient context for my question anymore. Overall, this implementation seems to only perform better than my baseline to a certain degree, and it bears the risk of providing unrelated information.

However, when I tried what happens if the user changes the topic between two utterances, the two-utterances-version seemed to work fine. I first asked about Feelgood’s phone number, then about how to take notes, and finally about signing up for courses. The LLM always answered correctly, and previous context information did not seem to lead to additional unrelated information in this case.

One more step would be to test what happens when Qdrant receives all user utterances in the context. I would expect it to work relatively well for my three-question test dialogue, but become noisy when the conversation continues for longer, as there would simply be too much – possibly already irrelevant – information in the context.

Question 2: Should you return a different number of query results?

In the regular part of Lab 1, I had already changed the number of query results from 5 to 3. This was done because I found that a limit of 3 led to better-suited answers compared to the earlier limit of 5.

To properly investigate how the query limit affected the output, I set the limit to 1, 3, and 5 and then asked the same four questions with each condition. The questions pertained to Feelgood’s phone number, taking notes, disability study support, and how to find the service center. I also changed the query back to the original one-latest-utterance condition.

Comparing the results for all four questions across the three conditions shows that limit: 1 can lead to exact and short answers, but also to insufficient answers that leave out information. Limit: 3 often produces more complete answers, and in one case even features redundant information in an answer. Limit: 5 leads to rather long answers that contain a lot of information that can be redundant and even irrelevant.

Depending on the question, different limits can lead to better results. For questions that can be answered with just a single piece of information (like the phone number), a limit of 1 was sufficient, but other questions (like the disability support) required a higher limit in order to retrieve all the necessary information and provide a full answer. The results also depend on the chunking of the documents. Relevant information might not be in one single chunk but be split across several chunks – in which case a higher limit might be needed to retrieve all of it (but also risks adding unnecessary information). Overall, a limit of 3 seems to be a good compromise in this case.